

Theme Park · One Function, Many Stalls — Arguments & Parameters — English Worksheet (Advanced)

Topic: One Function, Many Stalls — Arguments & Parameters · **Course:** Theme Park Creator · **Time:** about 50 minutes

This lesson you build a market **stall** with a **function**. A stall has more parts than a booth — a stand, posts and a roof.

So your function needs **more parameters**: some for **materials**, some for the **corners** that set where it goes and how big it is. Change the **arguments** you **call** it with, and one function builds a whole market row.

These are the blocks you use this lesson:

```
blocks.fill(BRICKS, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.HOLLOW)
blocks.fill(AIR, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.REPLACE)
blocks.fill(OAK_FENCE, pos(x1, 1, z1), pos(x1, 3, z1), FillOperation.REPLACE)
```

`blocks.fill(material, corner1, corner2, ...)` fills a box between two corners. `pos(x, y, z)` is a spot: `x` = left/right, `y` = up (height), `z` = forward/back. `FillOperation.HOLLOW` = only the outer **shell**. `FillOperation.REPLACE` = the **whole solid** box. `AIR` clears blocks — fill the inside of the shell with `AIR` to hollow it out.

1 · Meet Arguments and Parameters 🎁

A **parameter** is a named empty box you create when you **define** the function. An **argument** is the real value you send when you **call** it, matched **by position** — order decides which box each one lands in.

DEFINE → the names in () are PARAMETERS (empty named boxes):

```
def build_stall( stand , post , roof , x1 , x2 , z1 , z2 ):  
    └─ materials ─┘    └─── corners ───┘
```

CALL → the values in () are ARGUMENTS (real values sent in):

```
build_stall( BRICKS , OAK_FENCE , BRICKS , 2 , 10 , 2 , 6 )
```

Matched by POSITION, left to right:

stand ←	BRICKS	x1 ←	2	z1 ←	2
post ←	OAK_FENCE	x2 ←	10	z2 ←	6
roof ←	BRICKS				

Two things to notice: **BRICKS** is sent in **twice** (same material for stand and roof — fine), and the **corners** are numbers while the **materials** are words.

Why does matching by *position* mean the order of your arguments matters?

2 · Predict 🧠

Read each function. Write what you think happens.

```
def build_stall(stand, post, roof, x1, x2, z1, z2):
    blocks.fill(stand, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.HOLLOW)
    blocks.fill(roof, pos(x1, 4, z1), pos(x2, 4, z2), FillOperation.HOLLOW)

build_stall(BRICKS, OAK_FENCE, GLASS, 2, 10, 2, 6)
```

What material is the stand? What material is the roof? Which corners are the far ones?


```
def build_stall(stand, post, roof, x1, x2, z1, z2):
    blocks.fill(stand, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.HOLLOW)
    blocks.fill(post, pos(x1, 1, z1), pos(x1, 3, z1), FillOperation.REPLACE)

build_stall(BRICKS, OAK_FENCE, BRICKS, 10, 2, 2, 6)
```

The **x1** and **x2** arguments are **10** and **2** — the wide corner comes before the near one.
What might go wrong, or look flipped, compared to **2, 10** ?

3 · Spot the Bug 🐛

Each block is broken. Read what it should do, rewrite it so it works, then explain why the original was wrong.

Bug A — This should build a **brick stall with fence posts**. But it comes out with fence-material walls and brick posts — swapped.

```
def build_stall(stand, post, roof, x1, x2, z1, z2):  
    blocks.fill(stand, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.HOLLOW)  
    blocks.fill(post, pos(x1, 1, z1), pos(x1, 3, z1), FillOperation.REPLACE)  
  
build_stall(OAK_FENCE, BRICKS, BRICKS, 2, 10, 2, 6)
```

Hint: arguments land in the boxes **by position**. Which value went into `stand`?

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — The stand should be **hollow inside** so you can stand a shopkeeper in it. But the whole stand comes out as a **solid block** of bricks you cannot enter.

```
def build_stall(stand, post, roof, x1, x2, z1, z2):  
    blocks.fill(stand, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.REPLACE)  
  
build_stall(BRICKS, OAK_FENCE, BRICKS, 2, 10, 2, 6)
```

Hint: which `FillOperation` fills only the shell, and which fills the whole solid box?

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The roof should sit at the height you decide. But the roof always builds at the **same height**, because one corner value is ignored.

```
def build_stall(stand, post, roof, x1, x2, z1, z2):  
    blocks.fill(roof, pos(x1, 4, z1), pos(10, 4, 6), FillOperation.HOLLOW)  
  
build_stall(BRICKS, OAK_FENCE, BRICKS, 2, 20, 2, 12)
```

Hint: two of the corners are hard-coded numbers instead of the parameters `x2` and `z2`.

Write the fixed code:

Why was it wrong? Why does your fix work?

build_stall

```
def build_stall(stand, post, roof, x1, x2, z1, z2):
```

Your stall must include, in total:

- a **stand** — a hollow base between corners $(x1, z1)$ and $(x2, z2)$, cleared inside with **AIR** so it is open
- **four posts** — one rising from each of the four corners, made from the **post** material
- a **roof** — a flat top covering the stand, made from the **roof** material

Where the stall sits and **how big** it is must come from the four corner parameters — no fixed corner numbers inside.

Then **call your function two or three times** with different corners to line up a **market row**, for example:

```
build_stall(BRICKS, OAK_FENCE, BRICKS, 2, 10, 2, 6)
build_stall(STONE_BRICKS, DARK_OAK_FENCE, STONE_BRICKS, 12, 20, 2, 6)
```

```
build_stall
```

5 · Show Your Work 📷 🎥

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

Fill the blanks:

Today I built _____.

I built it using this code: _____.

In this code I used _____.

The hardest part was _____.

That part was hard because _____.

The most fun part was _____.

Something new I learned was _____.

Write your lines here, then say them in your video.

Submit ✓

Send this worksheet + your video to teacher on KakaoTalk.